TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 14: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `generate_graph`, `generate_points`, `generate_jobs`, `euclidean`, `distance_matrix`, `neighbors`, `is_vertex_cover`, `cut_value`.

The 12 problems are grouped into three themes:

- **Approximation Algorithms** (Problems 1–9): Vertex Cover via maximal matching, greedy Set Cover, greedy load balancing and LPT, greedy k-center, MST-based metric TSP, and the Knapsack FPTAS – each with an empirical check against a brute-force optimum or a provable bound.
- **Local Search** (Problems 10–12): Max-Cut local search (vertex flips), hill climbing with random restarts, and a simulated-annealing-style randomized search.
- **Empirical Ratios** (woven through Problems 2, 4, 5, 6, 8, 9, 11): measure the actual approximation ratio achieved and confirm it stays within the proven guarantee.

1 Approximation Algorithms

Problem 1 – Vertex Cover via Maximal Matching

Implement `vertex_cover_matching(graph)`, where `graph` is a tuple `(n, edges)`. Scan the edges; whenever an edge (u, v) has *neither* endpoint already in the cover, add *both* u and v to the cover. Return the cover as a `set` of vertices. Use `is_vertex_cover` from `starter_code.py` to sanity-check the result.

Example. On the path $0 - 1 - 2 - 3$ (edges $(0,1),(1,2),(2,3)$), scanning in order adds $\{0,1\}$ for edge $(0,1)$, skips $(1,2)$ (vertex 1 already in), and adds $\{2,3\}$ for $(2,3)$, giving cover $\{0,1,2,3\}$.

Problem 2 – Verifying the 2-Approximation for Vertex Cover

Implement `brute_force_min_vertex_cover(graph)` that returns the size of a *minimum* vertex cover by trying all 2^n vertex subsets (smallest first) and returning the first that covers every edge. Then implement `verify_vc_2approx(num_trials, n, p, seed)`: for each random graph (`generate_graph(n, p, seed=seed+trial)`), check that the matching cover (Problem 1) has size $\leq 2\times$ the optimum. Return `True` iff this holds on every trial. Intended for $n \leq 15$.

Problem 3 – Greedy Set Cover

A Set Cover instance is a universe `universe` (a set/frozenset of elements) and a list `subsets` of sets whose union is the universe. Implement `greedy_set_cover(universe, subsets)`: repeatedly pick the subset covering the *most* currently-uncovered elements, until all elements are covered. Return the list of chosen subset *indices*, in the order chosen.

Example. `universe = {1,2,3,4,5}`, `subsets = [{1,2,3}, {2,4}, {3,4}, {4,5}]`: greedy first picks index 0 (covers 3), then index 3 (covers $\{4,5\}$), giving `[0, 3]`.

Problem 4 – Greedy Set Cover vs. the Optimum

Implement `brute_force_min_set_cover(universe, subsets)` that returns the minimum number of subsets needed (try all sub-collections of size $1, 2, \ldots$ until one covers the universe). Then implement `set_cover_ratio(universe, subsets)` returning `greedy_size / opt_size` as a float. Verify on the small instances in the test block that greedy's size never exceeds the optimum by more than the H_n factor ($H_n = \sum_{k=1}^{n} 1/k$).

Problem 5 – Load Balancing: Greedy List Scheduling

Implement `greedy_list_scheduling(times, m)`: assign each job (in the given order) to a machine of currently minimum load. Return `(loads, makespan)` where `loads` is the list of m machine loads and `makespan` is their maximum. Then implement `verify_greedy_makespan(num_trials, n, m, seed)`: for each random instance (`generate_jobs(n, seed=seed+trial)`), check that the makespan is $\leq 2\times$ the lower bound $\max(\max_j t_j,\ \frac{1}{m}\sum_j t_j)$. Return `True` iff this holds on every trial.

Problem 6 – Longest-Processing-Time (LPT) Scheduling

Implement `lpt_scheduling(times, m)`: sort the jobs in *decreasing* order, then run greedy list scheduling (Problem 5). Return `(loads, makespan)`. Then implement `verify_lpt_makespan(num_trials, n, m, seed)`: check that the LPT makespan is $\leq \frac{4}{3}\times$ the lower bound $\max(\max_j t_j,\ \frac{1}{m}\sum_j t_j)$ on

every random instance. Return `True` iff so. (We use a small tolerance for floating-point comparisons.)

Problem 7 – Center Selection (Greedy Farthest-Point k-Center)

Implement `greedy_k_center(points, k, seed)`: pick a first center (deterministically, e.g. index 0, after optionally shuffling with `seed` for reproducibility); repeatedly add the point *farthest* from the current center set, until k centers are chosen. Return `(centers, radius)` where `centers` is the list of chosen point *indices* and `radius` $= \max_p \min_c d(p, c)$ is the covering radius. Use `euclidean` from `starter_code.py`.

Problem 8 – Metric TSP 2-Approximation via MST Preorder

Implement `mst_tsp_tour(points)`: build a minimum spanning tree of the complete Euclidean graph (Prim's algorithm is fine), then return `(tour, tour_length)` where `tour` is the preorder DFS order of the MST (a list of point indices, not repeating the start) and `tour_length` is the closed-tour length (summing consecutive distances and the wrap-around edge back to the start). Then implement `verify_tsp_2approx(num_trials, n, seed)`: for each random point set, check that `tour_length` $\leq 2 \times w(\text{MST})$. Return `True` iff so.

Problem 9 – Knapsack FPTAS (Rounding and Scaling)

Implement `knapsack_exact(profits, weights, capacity)` via the standard weight-indexed DP, returning the optimal total profit. Then implement `knapsack_fptas(profits, weights, capacity, epsilon)`: set $\mu = \varepsilon\, p_{\max}/n$, round each profit down to $\hat{p}_i = \lfloor p_i/\mu \rfloor$, solve exactly on the rounded profits (profit-indexed DP), and return the *true* total profit of the chosen items. Then implement `verify_fptas(num_trials, n, epsilon, seed)`: check that the FPTAS value is $\geq (1-\varepsilon)\times$ the exact optimum on every random instance. Return `True` iff so.

2 Local Search

Problem 10 – Max-Cut Local Search (Vertex Flips)

Implement `max_cut_local_search(graph, seed)`: start from a (seeded) random partition `side` (a list of 0/1 per vertex); while some vertex has more edges to its own side than to the other side, flip it (this strictly increases the cut). Return `(side, cut)`. Then implement `verify_local_optimum_half(num_trials, n, p, seed)`: check that the returned cut is $\geq m/2$ (where m is the edge count) on every random graph. Return `True` iff so. Use `cut_value` from `starter_code.py`.

Problem 11 – Hill Climbing with Random Restarts vs. Brute Force

Implement `brute_force_max_cut(graph)` that returns the maximum cut value by trying all 2^{n-1} partitions (fix vertex 0 on side 0 to avoid the symmetric double-count). Then implement `hill_climbing_restarts(restarts, seed)`: run `max_cut_local_search` from `restarts` different seeds and return the best cut found. Verify in the test block that, with enough restarts, the best cut found matches the brute-force optimum on the small test graphs, and that even a single restart never exceeds the optimum. Intended for $n \leq 12$.

Problem 12 – Simulated-Annealing-Style Randomized Max-Cut

Implement `simulated_annealing_max_cut(graph, seed, iterations, t_start, t_end)`: starting from a seeded random partition, on each iteration pick a random vertex and consider flipping it; if the flip improves the cut, accept it; otherwise accept it with probability $e^{-\Delta/T}$ where $\Delta \geq 0$ is the decrease in cut value and T is the current temperature (cooled geometrically from `t_start` to `t_end` over the iterations). Track and return `(best_side, best_cut)` – the best partition *ever* seen. Then verify that the annealing result is *never worse* than the plain hill-climbing local optimum (Problem 10) baseline on the tested seeded instances. (All randomness must be seeded so the test is deterministic.)

3 LeetCode Practice

After completing the problems above, practise this week's techniques – greedy approximation, binary-search-on-the-answer, and local search – on the following LeetCode problems. Difficulty is LeetCode's own label.

- Koko Eating Bananas – *Medium* – binary search on answer.
- Capacity To Ship Packages Within D Days – *Medium* – binary search on answer.
- Split Array Largest Sum – *Hard* – binary search on answer.
- Minimize Max Distance to Gas Station – *Hard* – binary search on answer.
- Minimize the Maximum Difference of Pairs – *Medium* – binary search + greedy.
- Maximum Performance of a Team – *Hard* – greedy + heap.
- Smallest Sufficient Team – *Hard* – set cover approximation.
- Minimum Number of Arrows to Burst Balloons – *Medium* – greedy cover.